

Python like Unladen Swallow, Python to C++ compilers like Shedskin, a Python-like compiled language, Wirbel, etc.

To enable Python to take full advantage of multi-core processors, there are solutions like the built-in *multiprocessing* module, *PProcess*, *Parallel Python*, etc. We will go on to show you how to employ these tools to pump steroids into Python, so it yields greater productivity, flexibility and performance.

In the first half of this two-part article, we explore *Python*, *Unladen Swallow*, *Shedskin* and *Wirbel*. In the next part, which will appear in the next issue of *LFY*, we will look at *multiprocessing*, *PProcess* and *Parallel Python*.

I used *Puppy Linux 4.2.1* and the *Ubuntu 9.10* 64-bit desktop edition to explore these tools and to test the source code presented in this article.

From now on, I'll use the terms 'C-like' or 'C-type' to identify C, C++ and other languages that compile to native code and offer excellent performance—it gets irksome to type 'C, C++, etc' multiple times, and you'll probably tire of reading that, too.

Why is Python slow?

When you compile and link a program in C-like languages, the entire program is converted into instructions that are natively executable by your hardware—the CPU. In contrast, running a Python program is mainly a two-step process. When you run a Python program, it is automatically converted into a platform-independent intermediate representation known as byte-code. These byte-code instructions are then interpreted through a native platform application known as the Python Virtual Machine. The official, reference, and most popular implementation of the Python virtual machine is *CPython*. It's obvious from the name itself that it is programmed in C. Every single byte-code instruction invokes a C routine, comprising multiple native instructions, to accomplish the task encoded in the byte-code. It's this process that introduces the execution delays.

Python is a dynamic programming language, where every data type is an object—even if it looks like a native data type to you. Type `i = 7`; `f = 3.14` at a Python interactive prompt, and then type `dir(i)`; `dir(f)` to see a listing of the various methods and attributes associated with `i` and `f`. In Python, you directly assign values to variables without declaring their types—only the values have a type in Python, not the objects. This is the opposite of static C-like languages, where you have to first declare the types of the native or abstracted objects, before manipulating them. Python also lets you bind different types of values to objects at runtime. Therefore, the Python runtime (VM) also has to deduce the types of these objects, and convert them into native data types, to finally build the native instructions that will run on the CPU. This type deduction and translation is another overhead that further slows down Python.

The various objects in Python are allocated and deallocated through a subsystem for automatic memory management known as the *gc* (Garbage Collector). This *gc* activity runs after a certain number of object allocations, to manage and free memory from unreachable objects. During the *gc* cycles, other

activities stop—so if the *gc* cycles are long, then those again introduce delays in execution of your Python code.

The Python virtual machine deals with a lot of global objects, and the external non-thread-safe native-code modules. The Python interpreter enforces a rule that only a single running thread modifies those global objects, and ensures the safety of other threads by granting a mutual exclusion lock to the scheduled thread. The lock is known as the GIL (Global Interpreter Lock). Though the design of the Python interpreter is simplified and it's easier to maintain in this way, it introduces yet another speed-reducing side-effect to Python programs. Due to the GIL, Python programs are unable to take advantage of modern multi-core processors that can run multiple threads of instructions in parallel. This means that even if you write your Python programs with multiple threads, the Python interpreter will only execute one thread at a time on a CPU core; the remaining cores are not used. That's another big performance loss. (This was just a simple picture of the GIL; it's one of the most-discussed topics in the Python community, and you could learn more about it in an excellent video presentation mentioned in the reference section.)

Psyco, a Python specialising compiler

The easiest option to speed up your Python apps, without significantly changing their code, and while still using the standard Python interpreter, is to use *Psyco*. This is a loadable Python module; you just *import* the *psyco* module, and add a line or two to speed up your existing Python code. *Psyco* is a mature piece of work that's been around for many years, but currently it is in maintenance-only mode, and is only available for 32-bit Python distributions on the x86 processor architecture. According to the author of *Psyco*, the current and future work related to it is being targeted towards a new Python implementation known as *PyPy*. *Psyco* claims to provide speed-ups in the range of 2 to 100 times, typically a 4x speed-up with the unmodified Python sources and the standard interpreter. It is available for multiple OS platforms: GNU/Linux, BSD, Mac OS/X and Windows.

Installing Psyco

Psyco is written in C, so you need standard build tools to build it. You also need the standard Python headers to install *Psyco* from its source tarball. Type the commands `sudo apt-get install build-essential` && `sudo apt-get install python-dev` in a text console on your Ubuntu 32-bit machine, to fulfil the dependencies. Now go to the *Psyco* project page, download its latest source tarball, and issue the commands `tar xzvf psyco-version-src.tar.gz` && `cd psyco-version`. Finally, issue the command `python setup.py install`, with `sudo` or `su` if you want to do a system-wide install. If everything goes well, check by running `python -c 'import psyco'`. If there's no error, you are ready to play with *Psyco*.

Quickly mod your Python app with Psyco

If you're impatient to put *Psyco* in your Python application without reading up about it, then add the lines of code below